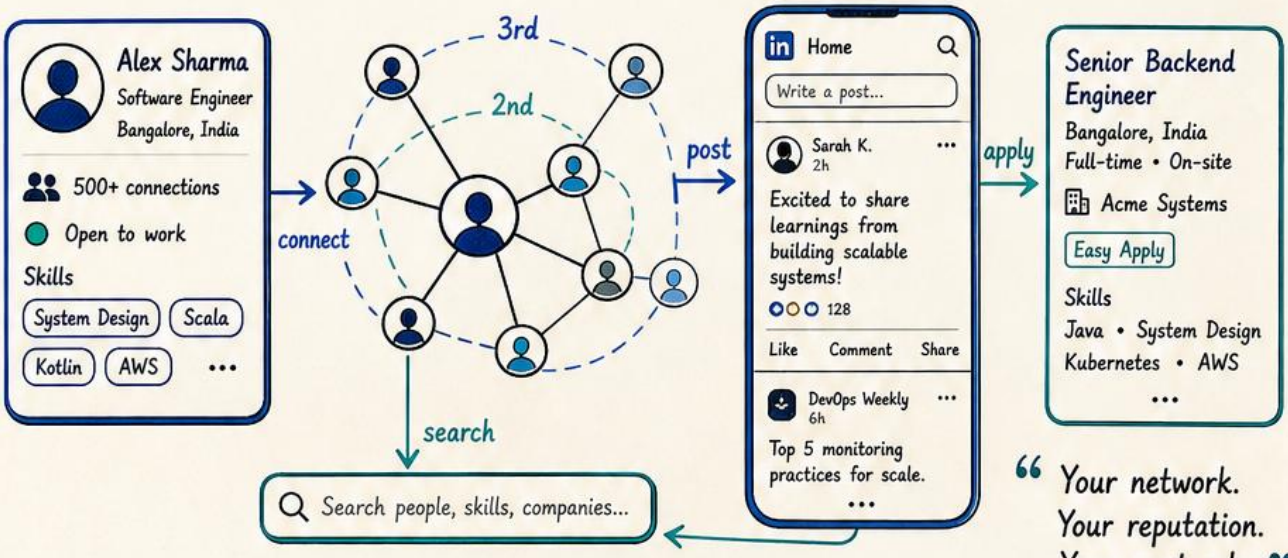
@ codedsoul_05

PROFESSIONAL NETWORK SD v1.0

LINKEDIN SYSTEM DESIGN

— HANDWRITTEN HANDBOOK —

Designing Profiles, Connections, Feed, People Search & Jobs



"Your network. Your reputation. Your next role."

Profile • Graph • Feed • People Search • PYMK • Jobs

What's Inside ?

1. Requirements & Scale
2. High-Level Architecture
3. Data Model
4. Core Entities & Relationships
5. Feed System Design
6. People Search Design
7. You May Know (PYMK)
8. Jobs & Applications
9. Messaging System
10. Ranking & Relevance

\+ Media, Search, Caching & System Design Q&A

LinkedIn by the Numbers

- 1B+ Members globally
- 30B+ Connections
- 15M+ Jobs listed
- Millions of posts daily
- Search-heavy platform (people, skills, companies)

The graph is the product.

Topics Covered

- ☑ Bidirectional Connection Graph
- ☑ Degrees (1st, 2nd, 3rd)
- ☑ Feed Fan-out & Fan-in
- ☑ Elasticsearch People Search
- ☑ You May Know (PYMK)
- ☑ Jobs, Applications & Matching
- ☑ Messaging & Notifications
- ☑ Ranking & Personalization
- ☑ Privacy, Blocking & Reporting
- ☑ Scalability, Consistency & Caching

Who is this for ?

- System Design Enthusiasts
- Backend / Full Stack Engineers
- PMs, Founders & Tech Leads
- Anyone building social platforms

"Twitter follows are one-way. LinkedIn connections are negotiated trust."

Connect With Intent. Rank With Context.

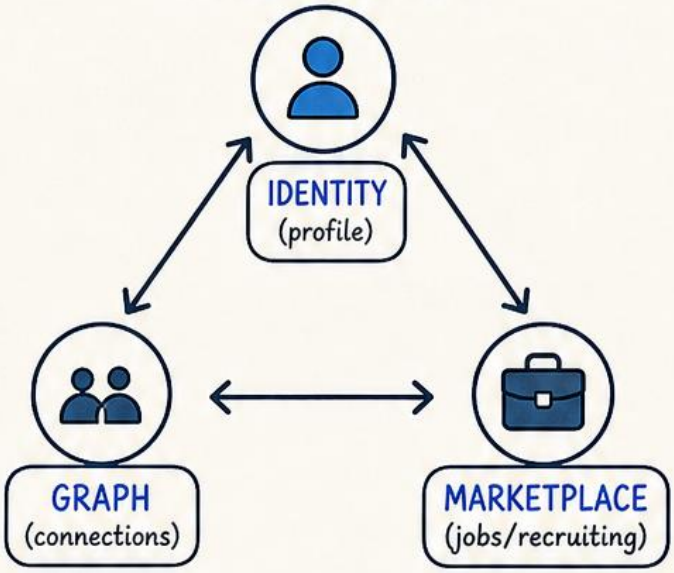
01 — REQUIREMENTS & SCOPE

Help professionals present identity, grow a trusted network, stay informed, and find opportunities.

CORE FEATURES

- Profiles
- Connection requests
- Home feed
- People search
- People You May Know
- Jobs search/apply
- Messaging
- Company pages
- Notifications
- Creator posts/articles (overview)

THREE PRODUCTS



Note: Feed is important, but search + graph are the moat.

FUNCTIONAL REQUIREMENTS

1. Identity

Create & manage professional profile (experience, skills, education, etc.)

2. Social graph

Send/accept connections, manage network, see relationships.

3. Content feed

Discover posts, articles, updates from your network and companies.

4. Opportunity matching

Find relevant jobs, apply, get discovered by recruiters.

NON-FUNCTIONAL REQUIREMENTS

- Low search latency (people, jobs, companies)
- Graph queries at scale (connections, mutuals, recommendations)
- Privacy & data protection by design
- High read throughput on profiles
- Eventual OK on feed delivery
- Strong-ish consistency on connection accept

OUT OF SCOPE

- ✕ Full LinkedIn Recruiter enterprise suite internals
- ✕ Learning courses deep dive
- ✕ Ads auction deep dive

INTERVIEW TIP

Don't only design Twitter-with-jobs. Lead with profile + bidirectional graph + people search.

Reputation graph meets opportunity marketplace. ★

02 — CAPACITY ESTIMATION

Profile and people-search reads dwarf posts. Graph edges are enormous.

ASSUMPTIONS

Members	DAU	Avg connections	Posts / day	Profile views / DAU / day	People searches / DAU / day	Job searches
= 1 Billion	= 100 Million	= 400 (mean; varies by user)	= 50 Million	= 10	= 2	Significant

PROFILE READS

100M × 10 = 1B views/day

≈ 11,600 / sec avg

Peak 50k+ / sec

— cache profiles hard

GRAPH

1B × 400 / 2 ≈ 200 Billion undirected edges (order-of-magnitude)

— store both directions for query patterns

FEED

50M posts/day

Fan-out similar celebrity/ influencer problem but milder than Twitter; hybrid still applies.

SEARCH QPS

100M × 2 = 200M searches/day

≈ 2,300 / sec avg

Peak 10k+ / sec against large people index

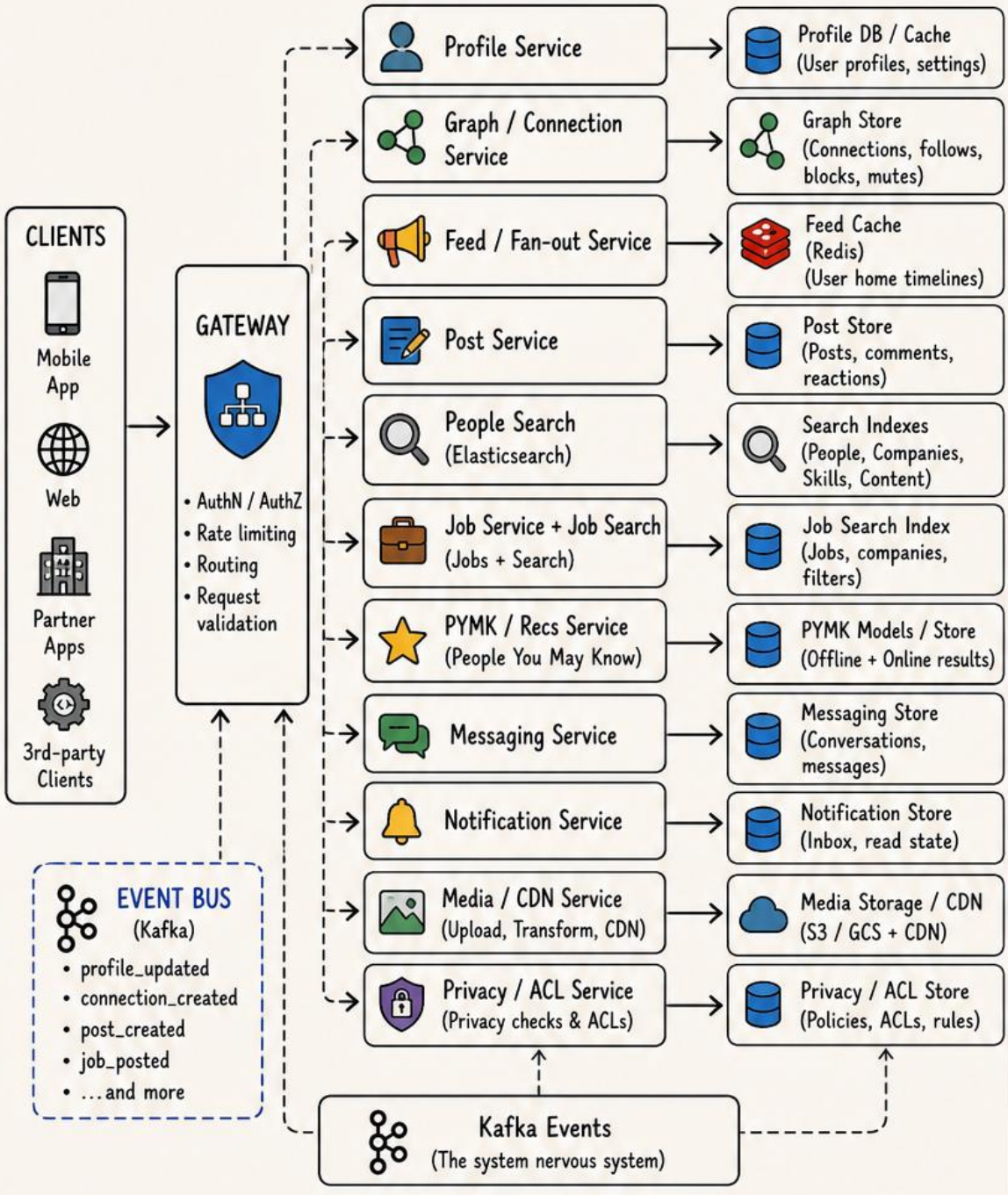
STORAGE

- Profiles: few KB each → multi-TB
- Edges: ~200 Billion (undirected)
  → store both directions → edges dominate
- Media: separate object storage

TAKEAWAY: cache profiles; shard graph by member_id; search cluster is first-class; feed is not the only hard part

Search and graph outrank pure feed volume. ★

03 — HIGH-LEVEL ARCHITECTURE



WHY

- Privacy checks on every view — enforce at service + data layer.
- Graph is separate from feed for scale and different access patterns.
- Search indexes are denormalised from profiles for fast lookups.
- Jobs have their own indexing & ranking pipeline.
- PYMK uses both offline (batch) signals and online (real-time) signals.

COMMON FLOWS (HAPPY PATHS)



TIP: Draw Privacy/ACL early — profiles are not public-by-default in the same way as tweets.

Profile, graph, search, feed — four pillars. ★

04 — APIs & CONTRACTS

Every read may be privacy-filtered. Never return raw graph without ACL.

PROFILE

- GET /v1/me
- GET /v1/members/{id}
- POST /v1/members/{id}/profilePicture

GRAPH

- POST /v1/invitations
- POST /v1/invitations/{id}/accept
- DELETE /v1/connections/{id}
- GET /v1/connections
- GET /v1/members/{id}/distance

FEED

- GET /v1/feed?cursor=
- POST /v1/posts
- POST /v1/posts/{id}/react

SEARCH

- GET /v1/search/people ?q=&filters=&cursor=
- GET /v1/search/jobs ?q=&filters=&cursor=...

JOBS

- POST /v1/jobs
- POST /v1/jobs/{id}/apply

MSG

- POST /v1/conversations/{id}/messages

INVITATION CREATE RESPONSE (JSON)

```
{
  "id": "inv_8f1a2c9d7e6b4a1f8c3d2e1a",
  "from": "urn:li:member:123",
  "to": "urn:li:member:456",
  "created_at": "2025-05-18T10:15:30.123Z",
  "status": "PENDING",
  "message": "Let's connect!",
  "invitation_url":
   'www/linkedin.com/invitations/inv_8f1a2c9d7e6b4a1f8c3d2e1a",
  "expires_at": "2025-06-17T10:15:30.123Z",
} "visibility": "PRIVATE"
```

INVITATION NOTES

- Invitations are rate limited.
- Duplicates return the same invitation id (idempotent).
- Expired invites cannot be accepted.

CURSOR PAGINATION

Use cursor for lists.
Pass cursor in ?cursor=.
Example:
GET /v1/feed?cursor=abc123&limit=20

RATE LIMIT INVITATIONS (SPAM)

Invitations are heavily rate limited.
Headers to watch:
x-rate-limit-limit
x-rate-limit-remaining
x-rate-limit-reset

IDEMPOTENCY ON APPLY

POST /v1/jobs/{id}/apply is idempotent.
Server guarantees exactly-one application per member per job.

FIELD PROJECTIONS FOR PROFILE (VIEWER-DEPENDENT)

Use fields= to request only what you need.
Returned fields vary by viewer permissions.

Same member_id returns different fields to self vs 1st vs recruiter vs public.

Viewer context is part of the API contract. ★

05 — DATA MODEL & STORAGE

Profile is a document. Connection is an edge with state. Feed stores activity IDs.

MEMBER

member_id	PK
name	
headline	
location	
industry	
visibility_settings	

EXPERIENCE

member_id	PK, FK
exp_id	PK
company_id	FK
title	
start	
end	

EDUCATION

member_id	PK, FK
edu_id	PK
school	
degree	
field_of_study	
start	
end	

SKILL

member_id	PK, FK
skill	PK
endorsements_count	

POST

post_id	PK
author_id	FK
type	(text|article|image|video|poll|...)
text	
media	(array)
created_at	

CONNECTION

a_id	PK, FK (member_id)
b_id	PK, FK (member_id)
status	(pending|connected|blocked)
created_at	

Store normalized a < b
PLUS direction for requests
OR store dual edges (a→b and b→a)

Critical: Bidirectional connection → often store two adjacency edges for fast "my connections" and mutual checks.

INVITATION

inv_id	PK
from	FK (member_id)
to	FK (member_id)
message	
status	(pending|accepted|canceled|expired)
created_at	

FEED_ITEM

member_id	PK, FK
activity_id	PK
score	(float)
ts	(timestamp)

Feed stores activity IDs with score & timestamp

COMPANY

company_id	PK
name	
industry	
size_range	
website	
created_at	

JOB

job_id	PK
company_id	FK
title	
location	
attrs	(json)
created_at	

MESSAGE

message_id	PK
conversation_id	FK
sender_id	FK
type	(text|attachment|...)
body	
created_at	
read_at	

CONVERSATION

conversation_id	PK
type	(1:1|group|inmail|...)
created_at	
participants	(array)

STORAGE & INFRASTRUCTURE

DOCMENT / PROFILE DB
- Primary: member profile (document)
- Also: experience, education, skills, company pages

Ex: MongoDB / DynamoDB

CACHE
- Profiles, timelines counters, relationships
- sessions, rate limits
- feed prefetch

Redis / Redis Cluster

GRAPH STORE / WIDE-COLUMN
- Connections as edges (dual edges or dir flag)
- Adjacency lists
- Fast mutual & distance queries

Neo4j / JanusGraph
Cassandra / HBase

POSTS STORE
- Posts, comments, reactions, shares
- Append-only time-ordered

Cassandra / ScyllaDB

SEARCH INDEX (ES)
- People & company profiles
- Jobs
  Keywords, skills, full-text search

Elasticsearch / OpenSearch

STREAMING
- User actions
- Writes, feeds, notifications, search updates

Kafka

OBJECT STORAGE
- Media files (images, videos, documents)

S3 / GCS

PARTITIONING & SHARDING

Partition by member_id.
Co-locate user's data for scale.

- Profiles & edges by member_id
- Feeds sharded by member_id
- Posts by time + member_id

Replication
Backups
Multi-AZ / Multi-Region

Pending invitation ≠ edge in the connected graph.

ACCESS PATTERNS

1. View profile as viewer
   - Apply visibility_settings
   - Merge blocking / hiding rules
2. List connections
   - Read adjacency edges
   - status = connected
3. Distance / mutual
   - Graph traversal
   - Mutual via intersection
4. Feed page
   - Read FEED_ITEM
   - by member_id
   - Apply score & ts
5. People search
   - Query ES index
   - Filter by skills, company, location, etc.

Documents for profile. Edges for trust. Indexes for discovery.

06 — MEMBER PROFILE

The profile is the product homepage — assembled per viewer with privacy.

PROFILE SECTIONS (WHAT WE SHOW)



READ PATH (VIEW PROFILE)

1. **Request**
   request(viewer_id, member_id)
2. **ACL Service**
   Decide field visibility based on viewer, member, and visibility rules.
3. **Fetch Profile Doc (+ Cache)**
   Read member profile document from store; check cache first.
4. **Attach Viewer-Specific Extras**
   - connection_degree
   - mutuals_count
   - can_message
5. **Return Projection**
   Send only allowed fields + extras to the viewer.

WRITE PATH (UPDATE PROFILE)

1. **Edit Section**
   Member edits a section.
2. **Validate**
   Validate content, permissions, and policy rules.
3. **Version Profile**
   Persist new version (append-only) and set as latest.
4. **Emit Event**
   Publish profile_updated (member_id, section, version).
5. **Update Systems**
   - Search index update
   - PYMK (People You May Know)
   - Cache invalidate

DESIGN & OPERATIONS NOTES

CACHE HOT PROFILES

Cache projected profiles per (member_id, viewer_tier) with short TTL.

Use stampede protection and stale-while-revalidate.

PARTIAL SECTION UPDATES

Update only the changed section.

Store sections independently to reduce write amplification.

HEADLINE & SEARCHABILITY

Headline, About, Skills, and Experience power search. Keep them fresh and structured.

CREATOR MODE EXTRAS

When creator mode is on, show follower count, content analytics, and audience insights.

COMPANIES & LINKED ENTITIES

Experience references companies. Keep company names, logos, and domains fresh via separate services.

⚠ Caching full profiles without viewer ACL leaks private fields.

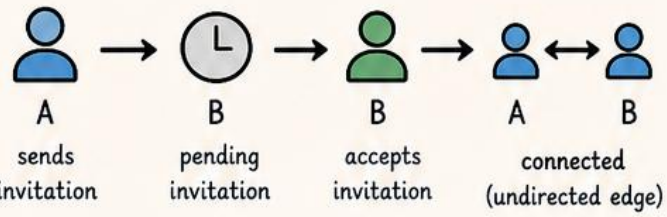
★ Cache carefully; project by viewer. ★

07 — CONNECTION GRAPH

KEY PAGE

Follow is subscribe. Connect is a handshake.

CONNECTION LIFECYCLE



Implement as two directed adjacency entries:

A → B and B → A

OPERATIONS

invite
Send a connection invitation.

accept
Accept a pending invitation.

ignore
Ignore (do nothing).

withdraw
Withdraw a sent invitation.

remove
Remove an existing connection.

block
Block a member (cuts everything).

LIMITS & SAFETY

- Daily invitation caps (e.g., per account tier).
- Cap total pending invitations (sent but not accepted).
- Spam invitations are a major abuse vector.
- Rate-limit and quality-score invitations.

ADJACENCY STORAGE

Adjacency lists sharded by member_id.

Shard(member_id = A)	Shard(member_id = B)
out : [B, C, D, ...]	out : [A, G, H, ...]
in : [E, F, ...]	in : [A, I, ...]

On accept: write both directions transactionally (A → B and B → A) or use a saga with reconcile to ensure dual adjacency.

QUERIES

isConnected(A, B)
Return true if A and B are connected.

listConnections(A)
Return all connections of A.

mutualFriends(A, B)
Return members connected to both A and B.

getDistance(A, B)
Shortest path length between A and B (via BFS / multi-source BFS).

VS TWITTER (X) GRAPH

	LinkedIn	Twitter (X)
Edge Type	Connection (mutual)	Follow (asymmetric)
Edge Rarity	Rarer	Very common
Signal	Higher (trust-based)	Lower (often passive)
Edge Creation	Requires acceptance	No approval needed
Abuse Risk	Spam invitations are a major vector	Spam follows, bots, mass follow
Edge Mutability	Remove / block cuts both directions	Unfollow is one-way

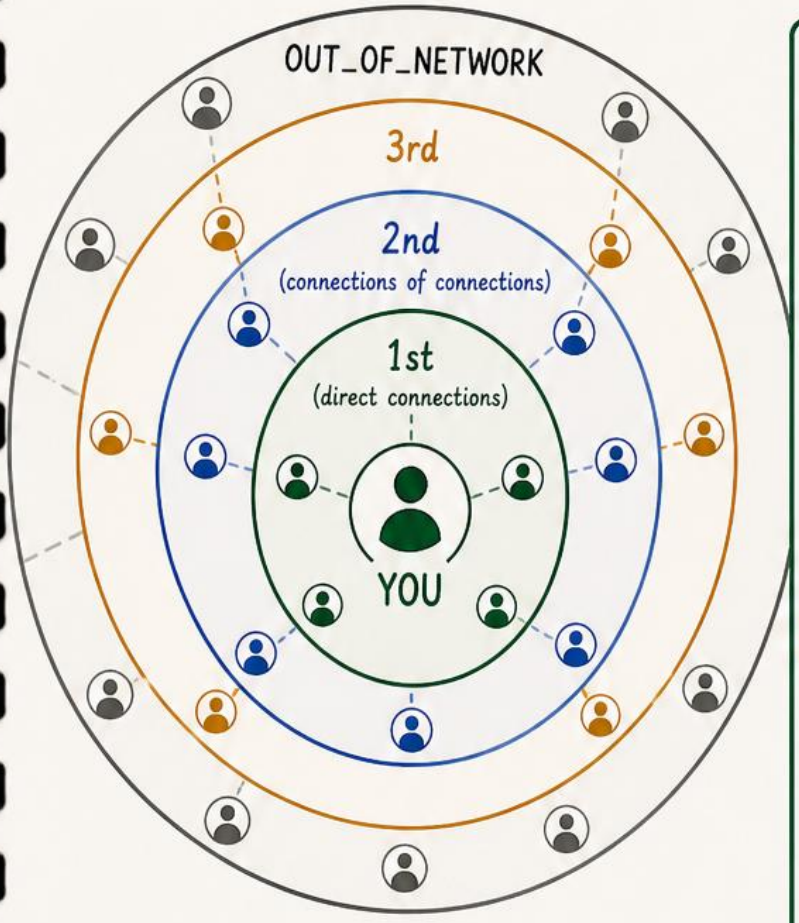
IMPLEMENTATION NOTES

- Store adjacency in sharded KV or wide-column store.
- Accept flow must be idempotent & safe under retries.
- Use anti-abuse signals: reputation, velocity, device/IP, content heuristics.
- Background jobs: expire old pending, reconcile dual edges, audit anomalies.
- Monitor: invitation rate, accept rate, block rate, abuse reports.

Invite → accept → dual adjacency. Block cuts everything.

08 — 1st / 2nd / 3rd DEGREE

Degree is a product primitive — it gates messaging, visibility, and ranking.



HOW TO COMPUTE DISTANCE

1 1st DEGREE

Edge lookup

▸ O(1) / O(log N)

2 2nd DEGREE

- Intersect neighbor sets
  OR
- Precomputed Bloom / bitmaps
- Online BFS (limited depth = 2)

▸ Typically fast and cacheable

3 3rd DEGREE

- Harder — more combinatorial
- Approximate with landmarks
- Precomputed neighborhoods
- Search-time signals

▸ Trade accuracy for speed

Cache distance for pairs commonly viewed.

MUTUAL CONNECTIONS

= Intersection of adjacency lists
(Optimize with smaller list iterate + hash set)



Show top mutuals (by relevance / recency / strength)



DO NOT RUN UNBOUNDED BFS ON EVERY PROFILE VIEW.

It's expensive, slow, and doesn't scale.

PRODUCT USES

WHO CAN INMAIL
Gated by degree (e.g., 1st/2nd/3rd or Open Profile only).

HOW PROFILE APPEARS
Degree influences profile surface, actions, and context.

PYMK (People You May Know)
Degree + signals drive suggestions and discovery.

FEED RELEVANCE BOOST
Closer degrees get higher baseline weight in feed ranking.

Exact 1st, efficient 2nd, approximate beyond.

09 — PROFESSIONAL FEED

Not only posts — jobs, birthdays, work anniversaries, news, and creator content mix.

ACTIVITY TYPES

- **Member posts** (text, images, videos, polls)
- **Articles** (long-form, newsletters)
- **Job changes** (if shared)
- **Company updates** (news, achievements)
- **Celebration cards** (birthdays, work anniversaries)
- **Sponsored** (ads, promoted content)

FEED BUILDING (READ PATH)

1. **Candidate activities** from network + followed entities
2. **Rank by** affinity / recency / engagement / professional relevance
3. **Filter for privacy** (visibility, blocks, muted, region rules)
4. **Paginate** (use cursors, not offsets)

DIFFERENT FROM TWITTER

- Lower post rate (people post less)
- Higher value per post (professional & durable content)
- More heterogeneous item types (not just short text)
- Ranking often prefers meaningful professional signals over raw recency/virality

KEY SYSTEM CARDS

- **Timeline cache** of activity ids (per user)
- **Hydrate** polymorphic items (many item types)
- **Seen / Dismiss state** (per user, per item)
- **Creator vs Consumer modes** (authoring vs reading experience)

Polymorphic activities. Professional relevance over pure virality.

10 — FEED FAN-OUT

Same push/pull tension as Twitter — influencers and company pages are the whales.

PUSH FAN-OUT ON WRITE

for normal authors

Push to connections' feed caches.

- Cheap reads
- Great for most posts
- Scales with limits

PULL FOR HIGH-CONNECTION / INFLUENCER / VIRAL COMPANY PAGES

Don't push to millions.
Let followers pull on read.

- Avoid write explosion
- Great for whales
- Read-time cost

HYBRID STRATEGY

Threshold by connection count + post frequency.

- Below threshold → PUSH
- Above threshold → PULL
- Borderline → Hybrid
- Adaptive over time

FOLLOWABLE ENTITIES BEYOND CONNECTIONS

Bring more voices into the candidate set.

- **COMPANY PAGES** — official updates, announcements
- **NEWSLETTERS** — publisher posts, curated content
- **CREATORS** — influencers, experts, individual voices
- **TOPICS / HASHTAGS** — trending topics, communities

WRITE PATH (PUSH)

Make the post visible via fan-out.

1. **PERSIST POST** — Store post & metadata (Author, ACL, visibility, etc.)
2. **KAFKA** — Publish post CreatedEvent to Kafka
3. **FAN-OUT WORKERS** — Workers consume, chunk adjacency lists, and enqueue fan-out tasks
4. **REDIS FEED LISTS** — Push post ID to followers' feed lists (Trim to max N per list)

Chunk adjacency to control memory and fan-out latency.
Use set/list with dedupe + LTRIM.

READ PATH (PULL + MERGE)

Assemble and rank the feed.

1. **PUSH LIST** — Get from user's Redis feed list (recent pushed posts)
2. **PULL ENTITIES** — Fetch from followables (influencers, companies, newsletters, creators, topics) on read
3. **RANKER** — Merge, de-dup, score, and rank (quality, recency, diversity)
4. **FEED** — Return top N for the user

Pull set is dynamic per user and can be cached.
Ranker applies business rules and ML signals.

CELEBRATIONS & SYSTEM ACTIVITIES

Some events are better generated online than fanned out.

- Birthdays
- Work anniversaries
- "Joined LinkedIn" anniversaries
- New connection milestones

Generated by scheduled jobs or real-time signals when read.

Shown inline in feed (or as special items) without pre-fan-out.

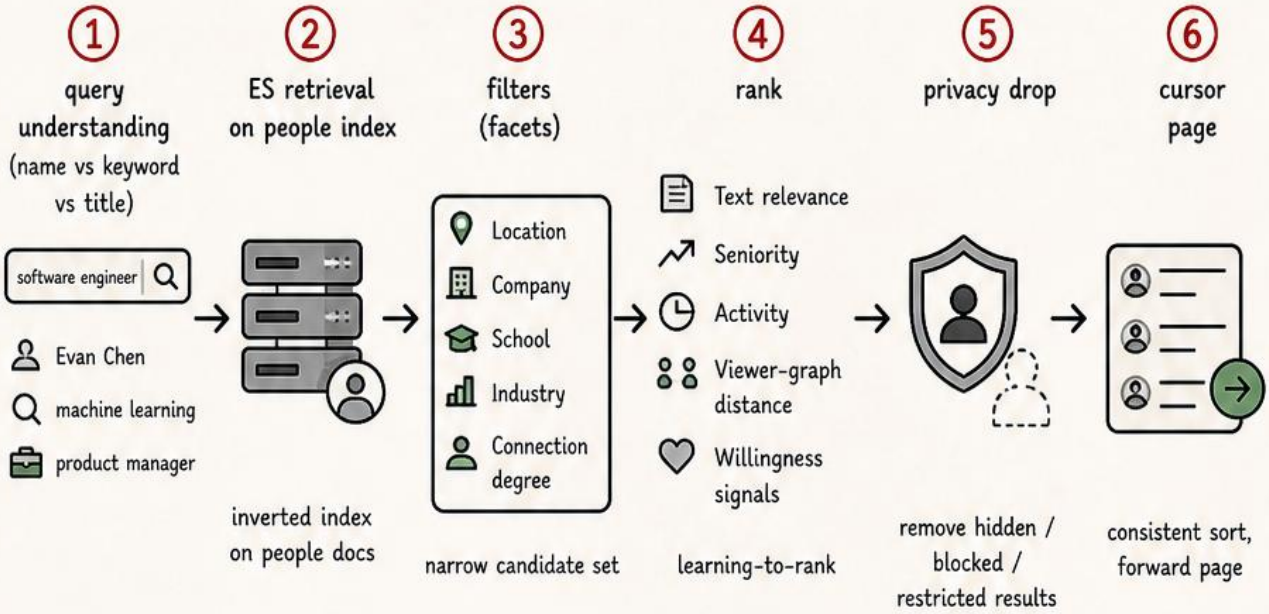
Push ordinary posts. Pull the megaphones. ★

11 — PEOPLE SEARCH

KEY PAGE

Recruiter-grade search is LinkedIn's hardest read path — relevance + filters + graph.

PEOPLE SEARCH PIPELINE (LINKEDIN)

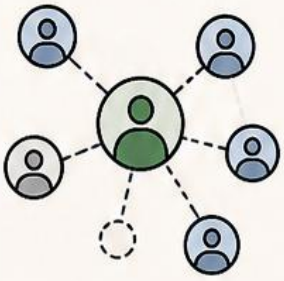


INDEX FIELDS (PEOPLE DOC)



- name
- headline
- skills
- experience titles/companies
- education
- location
- open_to_work flags (careful privacy)

GRAPH IN RANKING



- Boost 1st / 2nd degree connections
- Show mutuals (count & highlights)
- Filter by degree facet (1st, 2nd, 3rd+)

FRESHNESS



- profile_updated → near-real-time index
- not as instant as tweet search but must be fresh enough for job changes

SCALE & INFRASTRUCTURE



- sharded Elasticsearch cluster
- replica reads for scale
- caching of common recruiter queries (carefully)
- strict TTLs & invalidation

NEVER CACHE PERSONALIZED PEOPLE RESULTS GLOBALLY WITHOUT VIEWER KEY.

Retrieve people, filter by facets, boost by distance.

★ KEY PAGE

12 — PEOPLE YOU MAY KNOW (PYMK)

PYMK grows the graph — the best recommendations feel obvious in hindsight.

1. SIGNALS → CANDIDATES

- 2nd-degree (friends of friends) — strongest
- Same company / school / industry
- Co-view / co-apply / email address book imports (permissioned)
- Similar profile embeddings
- Geographic proximity

2. PYMK PIPELINE (PER MEMBER)

1. OFFLINE CANDIDATE GENERATION PER MEMBER
2. SCORE
3. FILTER
   - already connected
   - blocked
   - invitation limits
   - spam / low quality
4. STORE TOP-K
   - top-K
5. ONLINE RE-RANK WITH FRESH CONTEXT
   (recent activity, profile updates, engagement signals)
6. UI
   - Add
   - Add
   - Add

3. EXPLANATION IN UI

People you may know

Jane Doe — 12 mutual connections — Add

Why?
- You both work at Acme Corp
- 12 mutual connections
- You both follow Product Design
- You both viewed a job at Spotify

Display the top reasons (CONCISE & RELEVANT) that make the suggestion feel obvious.

4. AVOID

- Suggesting stalkery weak signals
- Over-representing celebrities
- Invite spam loops (keep it fresh, diverse & useful)

5. METRICS THAT MATTER

- **INVITE SEND RATE** — Are suggestions compelling enough to send?
- **ACCEPT RATE (PRECISION!)** — Do the right people accept? (quality over quantity)
- **DOWNSTREAM MESSAGING QUALITY** — Do connections lead to meaningful conversations?

★ FoF first, explain why, optimize accepts not clicks. ★

13 — JOBS MARKETPLACE

Two-sided matching: members seek roles; companies seek members.

ENTITIES

JOB POSTING



Role, requirements, location, comp (opt), remote, seniority

COMPANY



Organization posting jobs and managing recruiter seats

APPLICANT (MEMBER)



Searches, views, and applies to jobs

RECRUITER SEAT



Reviews applicants, manages pipeline, sends updates

SYSTEM FLOW

① POST JOB



Company or recruiter posts a job

② INDEX



Job indexed for search and matching

③ SEARCH / RECOMMEND



Members search or get recommended jobs

④ APPLY (EASY APPLY)



One-tap apply with resume and answers

⑤ NOTIFY RECRUITER



Recruiter notified of new applications

⑥ STATUS PIPELINE



Track applicants through the hiring pipeline

JOB SEARCH



Filters

- Title / Keywords
- Location
- Remote (On-site / Hybrid / Remote)
- Seniority (Intern, Entry, Mid, Senior, Lead, Director, ...)
- Salary (if present)
- Company

JOB RECOMMENDATIONS



- Member profile (skills, experience, education, location, preferences)
- Apply history & engagement signals
- Embedding similarity (job ↔ profile)
- Discard negatives (not interested, hide, unqualified feedback)

Score → rank → diversify → serve

APPLY PATH



- Idempotent apply
- Resume / profile attach
- Answers & additional info
- Rate limits & abuse protection
- Fraud job posts detection (spam, duplicate, scams)

FEED INTEGRATION



- Job alerts (saved searches, new matches)
- Suggested jobs shelves in feed
- Contextual placements (after profile views, skill additions, career actions)
- Personalized, timely, and non-intrusive

SCALE & PERFORMANCE



- Jobs index << People index (smaller corpus)
- High QPS for search & recommendation
- Low-latency reads, write-optimized ingestion
- Caching, sharding, replication
- Freshness: near real-time ingestion + backfill

ALERTS ARCHITECTURE



- Offline matching (batch)
- Candidate ↔ Job compatibility scoring
- Push via email / in-app / push notifications
- Frequency capping, quiet hours
- A/B tested strategies

TRUST & QUALITY



- Company verification & reputation
- Duplicate job detection
- Policy compliance & content filtering
- User reporting & feedback loops
- Hiring intent signals (reduce low-quality / ghost jobs)

KEY METRICS



- Job views
- Apply rate
- Save rate
- Interview rate
- Time to fill
- Quality of hire (downstream)
- Recruiter response time
- Member satisfaction / NPS

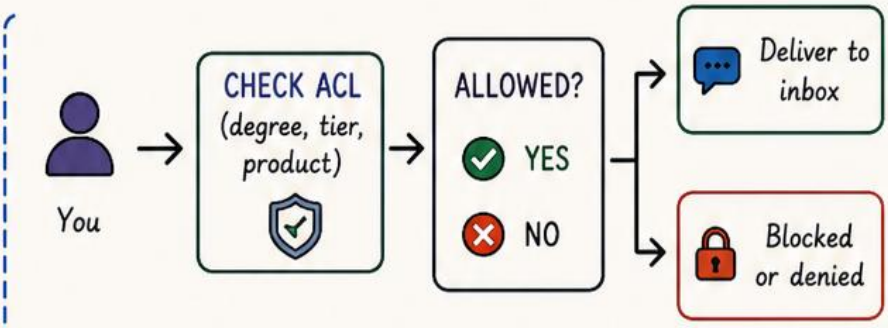
Index jobs, match profiles, apply once.

14 — MESSAGING & INMAIL

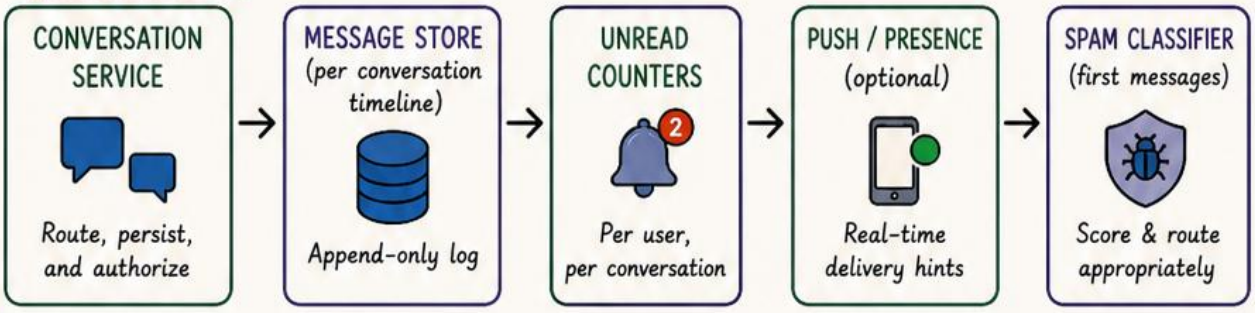
Messaging rights depend on degree and product tier — ACL before inbox.

1) RULES

- 1st degree can message
- InMail for outreach beyond
- Recruiter seats have limits
- Block / mute always wins



2) ARCHITECTURE



3) FAN-OUT (DELIVERY FLOW)



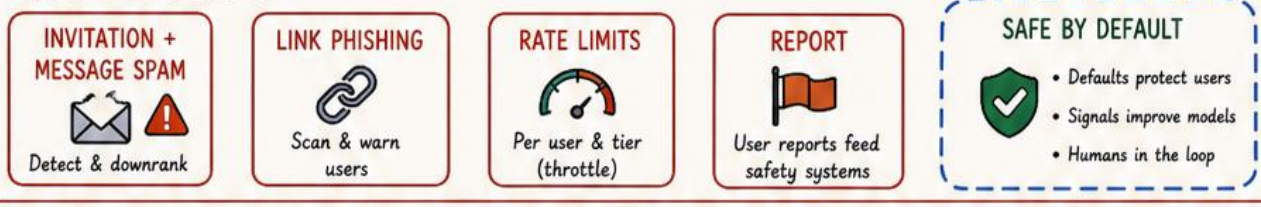
NOTES

- Ordered per conversation
- At-least-once delivery
- Idempotent operations

4) DATA & CONSISTENCY



5) ABUSE & SAFETY



Check degree/tier, then append-only conversation.

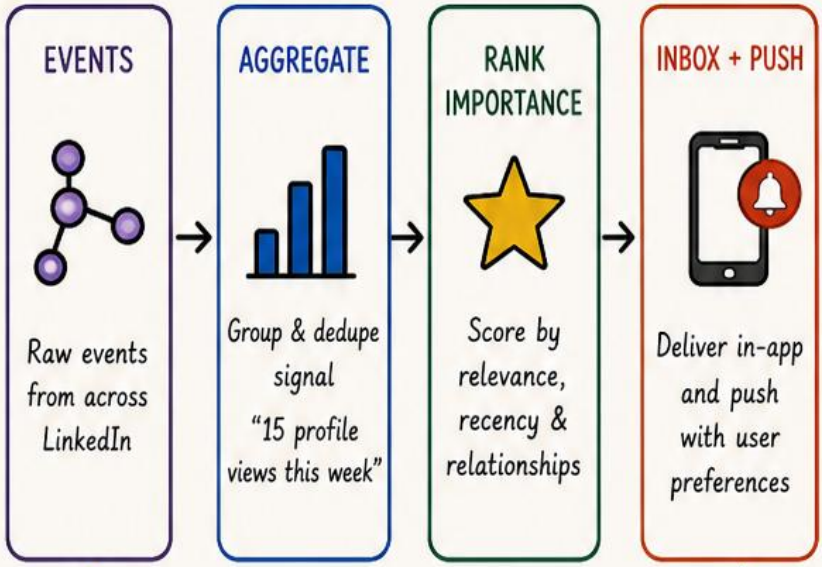
15 — NOTIFICATIONS & ENGAGEMENT

Professional notifications must feel useful — not like a social slot machine.

TYPES (WHAT WE NOTIFY)

- invitation accepted
- new connection
- profile views (aggregated)
- post reactions / comments
- job alerts
- birthday / anniversary
- mention
- InMail

NOTIFICATION PIPELINE



COUNTERS & PRIVACY



Profile view counts are privacy-sensitive.



Show approximate counts (e.g., ranges).



Respect each user's view visibility settings.

Aggregate views, respect settings, push sparingly.

16 — SCALING & CACHING

Shard members. Replicate search. Cache projections, not secrets.

1. SHARDS

MEMBERS
by member_id
(profile / graph / feed)
→ Member Shard 1 | Member Shard 2 | ... | Member Shard N

SEARCH CLUSTERS
separate clusters
(people vs jobs)
→ People Search Cluster | Jobs Search Cluster

KAFKA TOPICS
partitioned by
member_id / post_id
→ Member Events Topic | Post Events Topic

2. CACHES

PROFILE PROJECTIONS
- Key: (member_id, viewer_bucket)
- Careful: cache by viewer bucket
  OR
- Cache raw + apply ACL online (safer)

Profile Cache → Viewer Buckets

FEED IDS CACHE
- Redis sorted sets per viewer
- Timeline / home / network

→ Redis (Feed IDs) → Feed Service

DISTANCE CACHE
- Precomputed distances
- member ↔ business / event

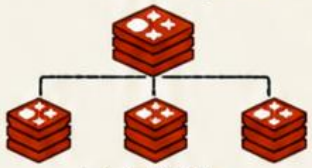
→ Distance Cache → Nearby Service

COMPANY PAGES
- Cache public company data
- Update via CDC / events

→ Company Cache → Company Service

3. HOT KEYS

- Celebrity / influencer members
- Viral posts
- Mega companies
- Isolate to prevent cascading load



Isolated Hot Key Tier (separate pool)

Rate limit • Pre-warm • Shard

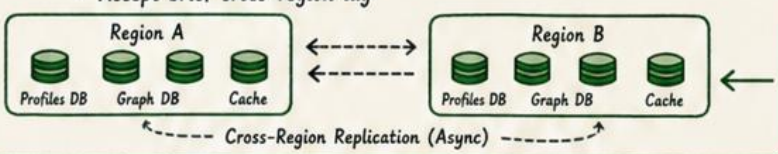
4. ONLINE vs OFFLINE

OFFLINE (Batch)
- PYMK / job recs
- Graph features
- Embeddings
- Precompute & store

ONLINE (Low Latency)
- Feed rank mix
- Real-time signals
- Lightweightt joins
- Personalization

5. MULTI-REGION

- Profiles & graph: geo-aware reads/writes
- Active-active or active-passive
- Replicate caches regionally
- Accept brief cross-region lag



GDPR & DATA RESIDENCY
- Keep data in-region
- Honor delete / export
- Pseudonymize where possible
- Audit & access controls

ACL after cache, or cache per viewer class.

17 — FAILURES & CONSISTENCY

Connection accept must not fork the graph. Feed can lag.

WHAT BREAKS	→ WHAT WE DO
Fan-out lag	delayed feed OK
Search index lag	stale headline briefly OK; job change should catch up fast
Accept connection partial write	saga/reconcile dual edges; repair job
Redis feed miss	rebuild from slower store
PYMK offline delay	show fallback popular/FoF online
Messaging down	keep graph/profile
ACL service down	fail closed on private fields

CONSISTENCY

✔ strong-ish for invitation accept;
eventual for feed/index; read-your-writes on own profile edits

RELIABILITY NOTES

- Embrace eventual consistency for feeds and indexes.
- Never fork the graph on connection accept.
- Repair dual edges automatically.
- Prefer showing something over errors.

WHAT TO MONITOR

- Fan-out lag (p95/p99)
- Search index lag
- Accept connection repair rate
- Redis feed miss rate
- PYMK offline delay rate
- Messaging down rate
- ACL service down rate
- Profile write RYW error rate

★ Fail closed on privacy. Repair the dual edges. ★

18 — TRUST, SPAM & PRIVACY

Fake recruiters and invite spam destroy the professional graph.

THREATS

- Mass invitations (connection spam)
- Fake profiles (stolen photos, made-up bios)
- Job scams (fake openings, advance fees)
- Scraping people search (harvesting profiles at scale)
- InMail phishing (malicious links, credential theft)
- Fake companies (impersonation, shell orgs)

CONTROLS

- Invite rate limits + accept-rate reputation
- Phone / email verification (strong identity signals)
- ML fake profile scores (risk scoring & review)
- Job scam classifiers (post & message screening)
- CAPTCHA / step-up (challenge on risk)
- Report / block (users, content, companies)
- Search API limits (rate limits & quotas)
- Field-level privacy (minimum data exposure)
- Export / download rights (your data, your control)

PRIVACY IN PRODUCTS

WHO CAN SEE WHAT?

- Email address: Only you
- Profile photo: Connections only (default)
- Activity (posts, reactions): Your connections (default)
- Messaging: Participants only

OPEN-TO-WORK GREEN BANNER VISIBILITY MODES

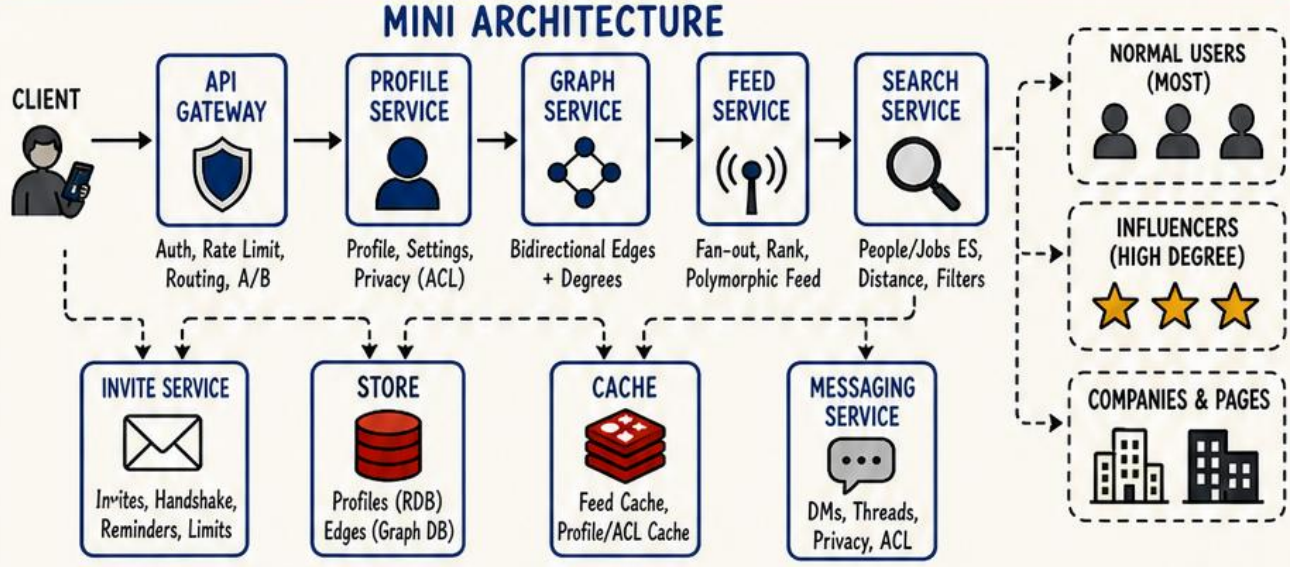
- Public to all LinkedIn members
- Recruiters only
- Off (not visible)

★ Throttle invites, verify identity, fail closed on private fields. ★

19 — RECAP & INTERVIEW CHEAT SHEET

ONE-LINER: Store rich profiles, grow a bidirectional graph with degrees, fan-out a professional feed, and make people/job search graph-aware under strict privacy.

MINI ARCHITECTURE



TOP 10 TALKING POINTS

1. Profile with rich data + ACL (viewer-aware).
2. Invite & handshake (request → accept).
3. Dual adjacency; (A,B) and (B,A) edges.
4. Degrees: 1st, 2nd, 3rd connections.
5. Hybrid feed fan-out: follow + network + topics.
6. Polymorphic feed: updates, posts, jobs, events.
7. People search with ES + graph distance.
8. PYMK (People You May Know) via FoF + signals.
9. Jobs is two-sided: seekers ↔ employers.
10. Messaging gated by degree + privacy rules.

VS TWITTER (KEY DIFFERENCES)

- Bidirectional graph vs unilateral follow.
- Search & Jobs are core moat (graph-aware).
- Lower post volume; richer, evergreen content.
- Privacy by default; viewer-aware projections.
- Degeres and distance unlock better recommendations.

QUICK Q & A

Q: How do you compute 2nd degree connections?
A: BFS up to depth=2 excluding already 1st degree and self; cache + TTL.

Q: How to handle celebrity/influencer fan-out?
A: Write-once to cache + push invalidation; use fan-out-on-write with sharding & rate limits.

Q: Cache vs ACL—how do you balance?
A: Cache public projections; enforce ACL per viewer at read time; short TTL + event invalidation.

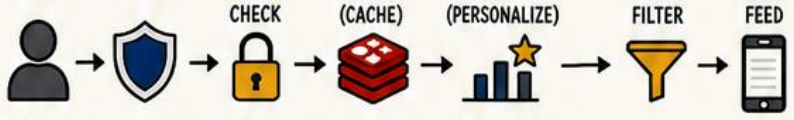
Q: PYMK—what metric to rank by?
A: Weighted score = FoF overlap + profile similarity + mutual groups + interaction signals; calibrate by accept rate.

Q: Connection accept—how to keep consistent?
A: Idempotent edges; single writer via partition key (min(idA,idB)); outbox + retries for notifications.

MISTAKES TO AVOID

- ✗ Designing Twitter-only (follow graph) model.
- ✗ Publicly caching private profile fields.
- ✗ Unbounded BFS (no depth limit) for suggestions.
- ✗ Invite/handshake APIs without rate limits.
- ✗ Ignoring Jobs & Search—biggest product leverage.

AT A GLANCE FLOW (READ PATH)



- Fan-out sources: connections, companies, topics, jobs, events.
- All reads are viewer-aware using ACL + projections.

HANDSHAKE GRAPH. VIEWER-AWARE PROFILE. SEARCH WITH DISTANCE.